

DIGITAL WAREHOUSE MANAGER

1. Project Overview :

Digital warehouse Manager is a small inventory and order management application built for a local warehouse. It solves the practical problem of tracking stock and orders manually: staff can manage products, customers, and orders from one place, while the system automatically prevents orders from exceeding available stock and stops prices or stock from going negative. It also gives the company visibility into which products sell best and basic sales analytics, calculated directly from the database rather than tracked by hand.

The application was built with Flask for the backend, chosen for how lightweight it is for a project of this size no unnecessary configuration or boilerplate. Rather than a separate frontend framework, the interface is rendered directly by Flask using server side HTML templates (Jinja2), which kept the project simple and let the focus stay on the SQL logic. SQLite was used as the database engine, accessed through Python's built in sqlite3 module with no ORM, so that every SQL query used by the application stays visible and readable in the source code rather than being generated behind the scenes.

2. Database Design:

The database has five tables: Category, Product, Customer, CustomerOrder, and OrderItem. Each product belongs to one category, and each order belongs to one customer. OrderItem is the table that connects orders to products: each row represents one product within one order, with foreign keys pointing to both CustomerOrder and Product. This structure lets a single order contain several different products, and lets the same product appear across many different orders.

The schema enforces several constraints beyond just the table structure. Every table has a primary key (id) that uniquely identifies its rows. Foreign keys link Product to Category, CustomerOrder to Customer, and OrderItem to both CustomerOrder and Product, ensuring every reference points to a row that actually exists. Customer.email has a UNIQUE constraint, so the same email cannot be registered twice. CHECK constraints on Product.price, Product.stock, OrderItem.quantity, and OrderItem.unit_price prevent any of these values from going negative. Finally, required fields such as names, prices, and foreign keys are all marked NOT NULL, so incomplete records cannot be inserted.

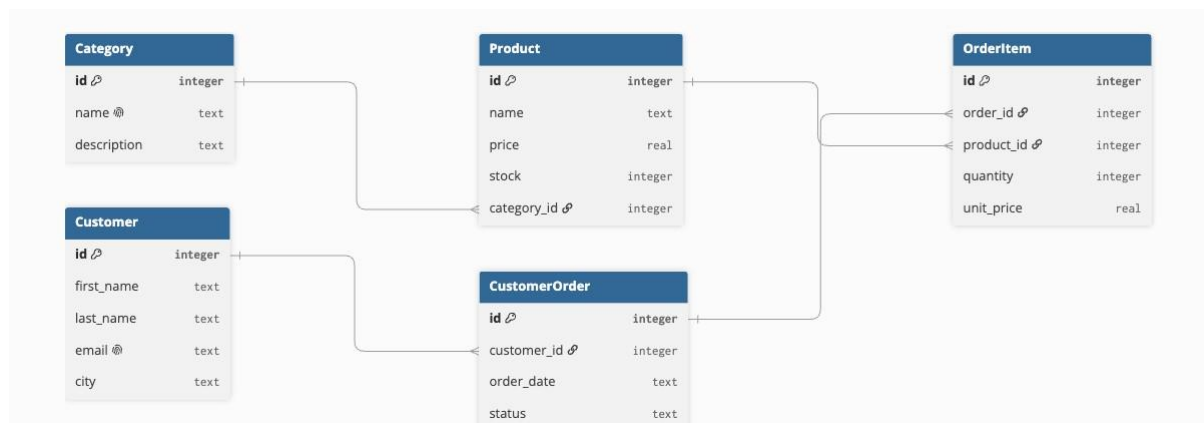


Figure 1: Entity relationship diagram of the Digital warehouse Manager database.

3. Application Architecture:

The application follows a simple three layer structure inside the `src/` folder. `db.py` handles the database connection: it opens a connection to `store.db`, enables foreign key enforcement, and configures results to be readable by column name. `app.py` contains all the Flask routes; each route runs its own SQL queries directly against the database and passes the results to a template for rendering. The `templates/` folder holds one HTML file per page, and all of them extend a shared `base.html` layout that provides the navigation bar and page styling.

The most important piece of logic is the order creation transaction. When a user submits the "New order" form, the application first inserts a new row into `CustomerOrder`. Then, for each product selected, it checks that the requested quantity does not exceed the current stock; if stock is sufficient, it inserts a row into `OrderItem` and decreases the product's stock accordingly. All of these steps are wrapped in a single database transaction. If any product in the order has insufficient stock, the application raises an error and calls a rollback, which cancels the entire transaction including the order row and any order items already inserted earlier in the same request. This guarantees the database is never left in a partially updated state, satisfying the transaction requirement in the project brief.

4. Application Pages:

4.1 Dashboard (/)

The Dashboard is the home page, showing four KPI cards (products, customers, orders, total revenue), a table of the five most recent orders, and the best selling product by total quantity sold. All values are calculated live from the database using `COUNT`, `SUM`, and `JOIN` queries nothing is hard coded.

Mini Store Manager

Dashboard

Products

Customers

Orders

Analytics

Dashboard

Products21

Customers12

Orders28

Revenue€468.10

5 most recent orders

Order	Customer	Date	Status	Total
#28	Bruno Bernard	2026-09-05 13:22:45	pending	€20.00
#27	Chloe Dubois	2026-09-05 13:19:55	pending	€9.60
#26	Emma Robert	2026-09-04 15:59:19	pending	€4.80
#12	Emma Robert	2026-08-23 19:41:00	pending	€1.30
#24	Gabrielle Durand	2026-08-20 10:24:00	paid	€31.80

Best-selling product

Chocolate Muffin — 17 units sold

4.2 Products(/products)

This page lists all products with their category, price, and stock. It supports searching by name, filtering by category, and sorting by name or price, all through parameterized SQL queries. Products with stock below 5 units are visually flagged. Full CRUD is implemented: products can be created, edited, and deleted, with server side validation preventing negative prices or stock.

Mini Store Manager Dashboard Products Customers Orders Analytics				
Products				
Search by name All categories Sort: Name Apply + New product				
Name	Category	Price	Stock	
Almonds 200g	Snacks	€4.80	10	Edit Delete
Baguette	Bakery	€1.20	30	Edit Delete
Butter 250g	Dairy	€2.40	0 (low stock)	Edit Delete
Cheddar Cheese 200g	Dairy	€3.60	18	Edit Delete
Chocolate Muffin	Bakery	€1.90	20	Edit Delete
Cinnamon Roll	Bakery	€2.50	4 (low stock)	Edit Delete
Cold Brew Coffee 330ml	Beverages	€2.80	25	Edit Delete
Croissant (pack of 4)	Bakery	€4.20	3 (low stock)	Edit Delete
Dark Chocolate Bar	Snacks	€2.60	33	Edit Delete
Digestive Biscuits	Snacks	€1.80	3 (low stock)	Edit Delete
Dish Soap 500ml	Household	€2.30	28	Edit Delete
Espresso Beans 250g	Beverages	€8.90	40	Edit Delete
Greek Yogurt 500g	Dairy	€2.90	22	Edit Delete
Green Tea Box (20 bags)	Beverages	€3.50	60	Edit Delete
Mozzarella 125g	Dairy	€2.10	27	Edit Delete
Orange Juice 1L	Beverages	€2.20	35	Edit Delete
Potato Chips 150g	Snacks	€2.00	45	Edit Delete
Sourdough Bread	Bakery	€3.40	15	Edit Delete
Sparkling Water 1.5L	Beverages	€1.10	80	Edit Delete
Whole Milk 1L	Dairy	€1.30	50	Edit Delete
coca	Beverages	€2.00	10	Edit Delete

4.3 Customers (/customers)

The Customers page shows a searchable list with each customer's order count and total amount spent, calculated using LEFT JOIN so that customers with zero orders still appear correctly with a count of 0. Clicking a customer opens a detail view with their full order history. As an additional feature beyond the brief's requirements, customers can also be created and deleted from this page.

Mini Store Manager Dashboard Products Customers Orders Analytics					
Customers					
+ New customer Search by name... Search					
Name	Email	City	Orders	Total spent	
Bruno Bernard	bruno.bernard@example.com	Paris	3	€63.40	Delete
Chloe Dubois	chloe.dubois@example.com	Lyon	2	€29.20	Delete
Gabrielle Durand	gabrielle.durand@example.com	Toulouse	2	€57.40	Delete
Karim Laurent	karim.laurent@example.com	Paris	4	€70.10	Delete
Hugo Leroy	hugo.leroy@example.com	Paris	3	€42.40	Delete
Alice Martin	alice.martin@example.com	Lille	1	€10.50	Delete
Lea Michel	lea.michel@example.com	Lyon	3	€49.60	Delete
Ines Moreau	ines.moreau@example.com	Lyon	2	€19.10	Delete
Farid Petit	farid.petit@example.com	Paris	4	€55.00	Delete
Emma Robert	emma.robert@example.com	Lille	3	€26.90	Delete
Julien Simon	julien.simon@example.com	Nantes	1	€44.50	Delete
David Thomas	david.thomas@example.com	Lyon	0	€0.00	Delete

4.4 Orders (/orders)

This page lists every order with its calculated total. A "New order" form allows the user to select a customer and one or more products with quantities; submitting the form creates the order, its order items, and updates stock as a single transaction, refusing the order if requested quantities exceed available stock.

[+ New order](#)

Orders

Order	Customer	Date	Status	Total	
#28	Bruno Bernard	2026-09-05 13:22:45	pending	€20.00	View details
#27	Chloe Dubois	2026-09-05 13:19:55	pending	€9.60	View details
#26	Emma Robert	2026-09-04 15:59:19	pending	€4.80	View details
#12	Emma Robert	2026-08-23 19:41:00	pending	€1.30	View details
#24	Gabrielle Durand	2026-08-20 10:24:00	paid	€31.80	View details
#6	Hugo Leroy	2026-08-18 10:59:00	paid	€17.70	View details
#18	Lea Michel	2026-08-11 12:41:00	paid	€38.00	View details
#5	Farid Petit	2026-07-20 13:51:00	pending	€9.50	View details
#11	Farid Petit	2026-07-06 14:22:00	paid	€4.10	View details
#23	Bruno Bernard	2026-07-05 19:10:00	shipped	€3.80	View details
#17	Emma Robert	2026-07-03 12:58:00	cancelled	€20.80	View details
#22	Chloe Dubois	2026-06-17 16:05:00	shipped	€19.60	View details
#10	Bruno Bernard	2026-06-13 13:29:00	shipped	€39.60	View details
#16	Alice Martin	2026-06-08 09:51:00	paid	€10.50	View details
#4	Farid Petit	2026-06-04 10:24:00	pending	€31.80	View details

4.5 Analytics (/analytics)

The Analytics page presents six SQL-driven reports: revenue by category, the top 5 customers by spending, products that have never been ordered, orders above the average order value, low-stock products, and total revenue by month.

Mini Store Manager

Dashboard

Products

Customers

Orders

Analytics

Analytics

A. Revenue by category

Category	Units sold	Revenue
Beverages	57	€183.70
Snacks	34	€108.60
Dairy	39	€99.30
Bakery	28	€62.70
Household	6	€13.80

B. Top 5 customers

Customer	Orders	Total spent
Karim Laurent	4	€70.10
Bruno Bernard	3	€63.40
Gabrielle Durand	2	€57.40
Farid Petit	4	€55.00
Lea Michel	3	€49.60

C. Products never ordered

Product
Sourdough Bread

D. Orders above average value

Average order value: €16.72

Order	Customer	Date	Total
#2	Julien Simon	2026-04-09 09:48:00	€44.50
#10	Bruno Bernard	2026-06-13 13:29:00	€39.60
#18	Lea Michel	2026-08-11 12:41:00	€38.00
#1	Karim Laurent	2026-03-23 17:26:00	€36.90
#4	Farid Petit	2026-06-04 10:24:00	€31.80
#24	Gabrielle Durand	2026-08-20 10:24:00	€31.80
#21	Gabrielle Durand	2026-05-19 15:23:00	€25.60
#17	Emma Robert	2026-07-03 12:58:00	€20.80
#9	Karim Laurent	2026-05-08 13:05:00	€20.20
#28	Bruno Bernard	2026-09-05 13:22:45	€20.00
#22	Chloe Dubois	2026-06-17 16:05:00	€19.60
#25	Hugo Leroy	2026-03-17 13:35:00	€19.10
#6	Hugo Leroy	2026-08-18 10:59:00	€17.70

E. Low stock products

Product	Stock
Butter 250g	0
Croissant (pack of 4)	3
Digestive Biscuits	3
Cinnamon Roll	4

F. Sales by month

Month	Revenue
2026-03	€80.20
2026-04	€66.10
2026-05	€58.90
2026-06	€101.50
2026-07	€38.20
2026-08	€88.80
2026-09	€34.40

5. SQL Focus: Two Advanced Queries:

```
sql
WITH OrderTotals AS (
  SELECT o.id, o.order_date,
         c.first_name || ' ' || c.last_name AS customer_name,
         SUM(oi.quantity * oi.unit_price) AS total
  FROM CustomerOrder o
  JOIN Customer c ON c.id = o.customer_id
  JOIN OrderItem oi ON oi.order_id = o.id
  GROUP BY o.id
)
SELECT id, order_date, customer_name, total
FROM OrderTotals
WHERE total > (SELECT AVG(total) FROM OrderTotals)
ORDER BY total DESC
```

5.1 Orders above the average order value (CTE + subquery)

This query uses a Common Table Expression (CTE) to first calculate the total value of every order in one place, giving that result set the name `OrderTotals`. It then reuses `OrderTotals` twice: once as the main table being selected from, and once inside a subquery that calculates the average of all order totals. Without the CTE, the join and aggregation logic would need to be written out twice once for each usage making the query longer and harder to maintain.

```
sql
SELECT p.id, p.name
FROM Product p
LEFT JOIN OrderItem oi ON oi.product_id = p.id
WHERE oi.id IS NULL
```

5.2 Products that have never been ordered (LEFT JOIN ... IS NULL)

A `LEFT JOIN` returns every row from the left table (`Product`) regardless of whether a matching row exists in the right table (`OrderItem`). When no match exists, all of `OrderItem`'s columns come back as `NULL` for that row, including its primary key, `oi.id`. Filtering on `oi.id IS NULL` therefore isolates exactly the products that were never matched to any order item in other words, products that have never been sold.

6. Conclusion:

The main difficulty in this project was Getting the error handling right, so that a partial failure never left the database in an inconsistent state, required careful use of BEGIN, commit, and rollback.

The application has some intentional limitations given its scope: it has no authentication, assumes a single currency, and does not support multiple stock locations.